📍 Part 1 — JSON Parsing

Q1:
You are given a JSON file called `coords.json`.
Inside the file, there is an array containing many objects, each representing a coordinate:

```
[
  {"lat": 37.123, "lng": -122.456},
  {"lat": 37.124, "lng": -122.457},
  ...
]
```

Task:

1. Load this JSON file from disk
2. Convert it into an in-memory object (`list` or Java class)
3. Print the first **10 coordinates**
4. Return them so they can be reused in later questions

🗺️ Part 2 — Send HTTP Request & Save the PNG

Q2:
You are given a **URL endpoint** (e.g., `https://internal.stripe.com/maps/render`) and a pre-written JSON request body.
The request body already contains:

- a center coordinate
- map zoom level
- style
- some default polyline path data

Task:

1. Use **HTTP POST** to send the request
2. The server will return a **PNG image**
3. Save the response as a file, e.g. `map.png`
4. Opening the file should display a basic map of the area (Stripe HQ region)

Python hint: `requests.post(url, json=payload)`
Java hint: `OkHttpClient`, `RequestBody.create()`, `Jackson` for JSON

🛣️ Part 3 — Modify the Request to Include Your Coordinates

Q3:
Now send another POST request to the **same URL**, same request structure as Question 2.

But this time:

- Replace the polyline/path data in the request
- Insert the coordinates you extracted in Question 1
- These will be drawn as a colored route from the JSON points → Stripe HQ

Task:

1. Modify the JSON payload to include your coordinates
2. Choose any color / stroke width (or use default)
3. POST the modified JSON to the API
4. Save the resulting image as `map_with_path.png`
5. When you open it, it should look exactly like Question 2's map, **but with the path drawn**

🔥 Part 4 — Similar Variant (Optional)

Q4:
Another HTTP variation:

- Maybe add a second path
- Or change colors dynamically per coordinate
- Or draw markers on each coordinate

You will reuse the same code from Q2 + Q3.

🔥 Part 5 — Another Similar Variant (Optional)

Q5:
Something like:

- Compute the distance between coordinates
- Or send only part of the coordinates
- Or draw a polygon instead of a path

Again, reuse your existing JSON loading and request-sending code.